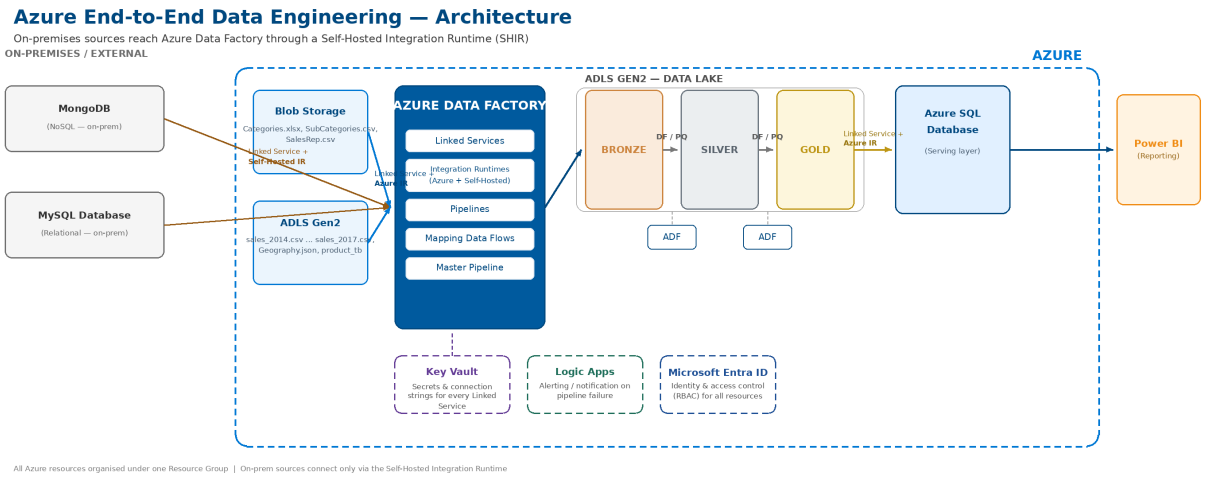
Azure End-to-End Data Engineering Project

Batch ETL Pipeline built on the Medallion Architecture (Bronze → Silver → Gold) using Azure Data Factory



ADLS GEN2	AZURE DATA FACTORY	KEY VAULT	AZURE SQL DB

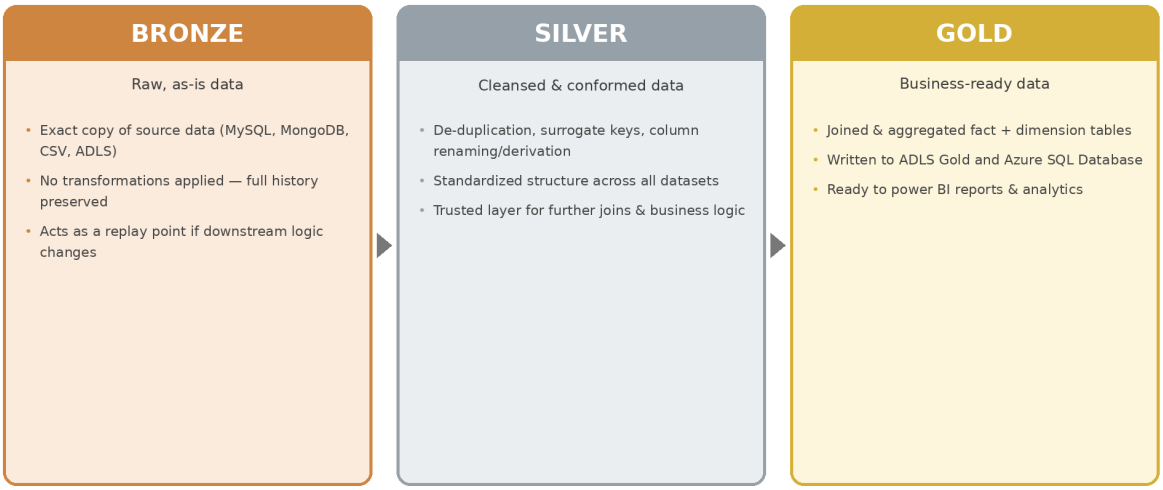
MONGODB (ON-PREM)	MYSQL (ON-PREM)

A hands-on project covering data ingestion from multiple heterogeneous sources, secret management, orchestration, and layered transformation — all the way to business-ready tables in Azure SQL Database.

Four source systems feed into Azure Data Factory. Blob Storage and ADLS Gen2 are native Azure services, so they connect over a standard **Linked Service + Azure Integration Runtime**. MongoDB and MySQL, however, are **on-premises / external databases** — they connect through a **Self-Hosted Integration Runtime (SHIR)**, which lets Data Factory securely reach systems that aren't hosted in Azure. From there, ADF orchestrates ingestion into Bronze, progressive transformation into Silver, and final joins/aggregation into Gold and Azure SQL Database, which feeds Power BI. Azure Key Vault secures every credential used along the way.

What is Medallion Architecture?

Medallion Architecture is a data design pattern used in modern data lakes to incrementally improve the structure and quality of data as it flows through three layers — **Bronze, Silver, and Gold**. Instead of transforming raw data into a final report in one giant step, each layer has a single clear responsibility. This makes pipelines easier to debug, reprocess, and trust.

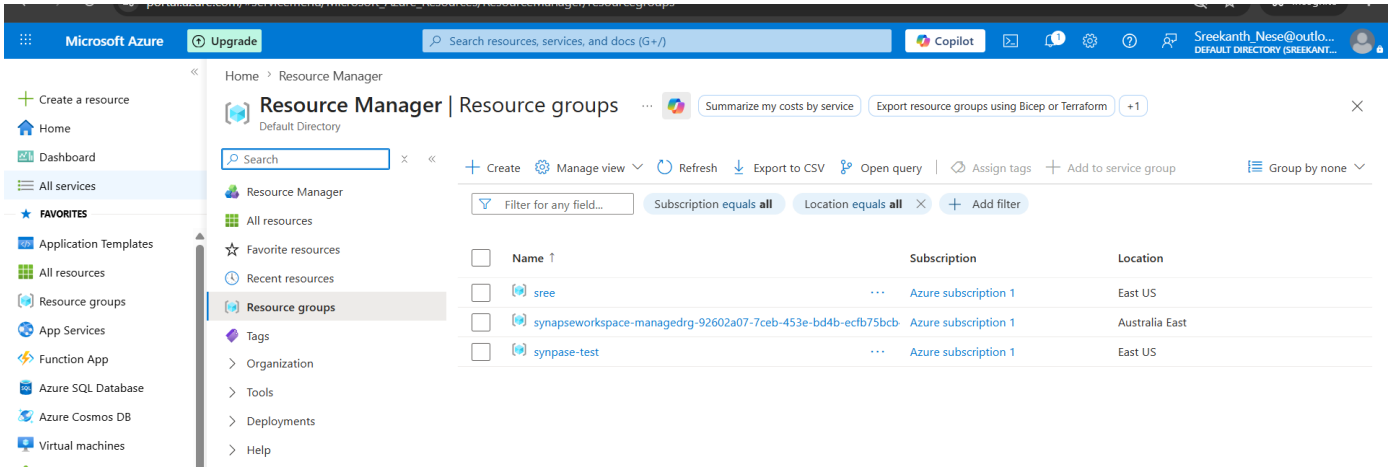


Bronze	Raw landing zone. Data is copied exactly as it arrives from source systems (databases, files, APIs) with no changes. It preserves full history so any step downstream can be reprocessed from scratch if logic changes.
Silver	Cleansed and conformed zone. Duplicates are removed, surrogate keys are generated, columns are renamed/derived, and datatypes are standardized — turning messy raw data into a trusted, query-ready shape.
Gold	Business/aggregated zone. Silver tables are joined together and aggregated into the fact and dimension tables that reporting tools and business users actually consume — typically also mirrored into a relational database for BI tools.

Why it matters: *each layer is independently testable, storage is cheap so keeping raw history is fine, and if business logic changes you only have to re-run Silver/Gold — not re-pull data from the original source systems.*

Azure Resource Group

Step 1 — Provisioning the Resource Group



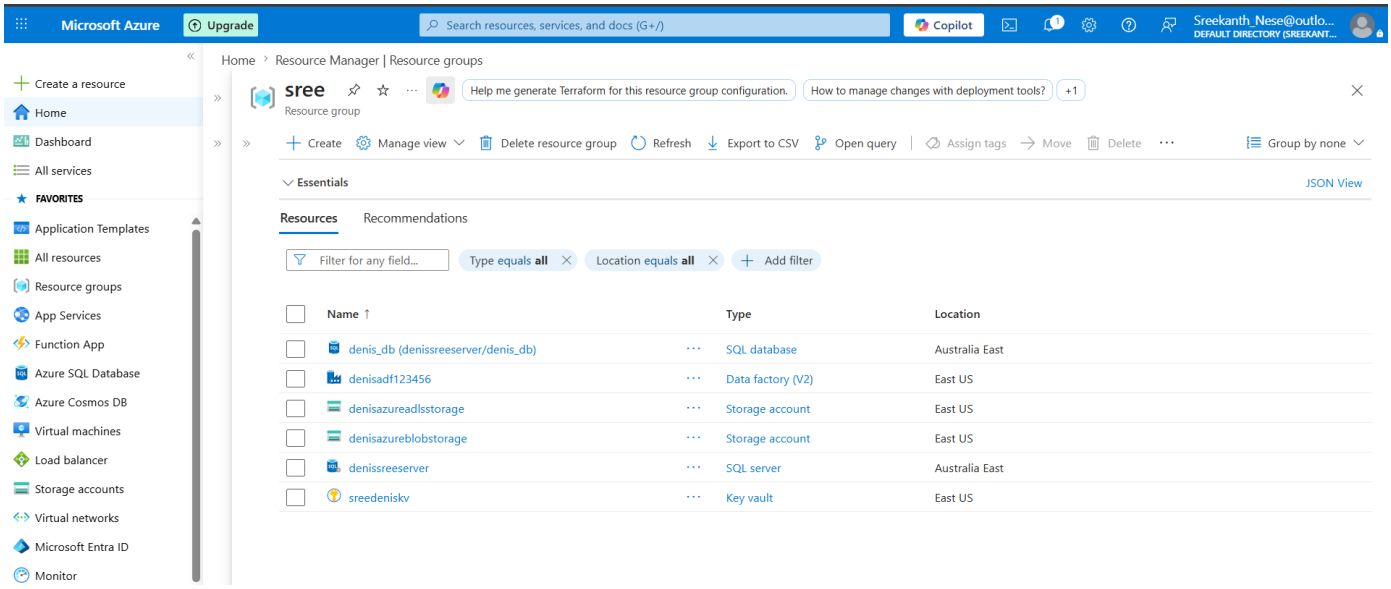
Every component of this project — storage, orchestration, database, and security — lives inside a single Azure Resource Group. Grouping resources this way keeps billing, access control, and lifecycle management simple: the entire environment can be monitored, scaled, or deleted as one unit.

How it's used: Used as the logical container for all other services in this build.

- Resource group: **sree**
- Resources deployed across East US and Australia East
- Naming convention: 'denis' / 'sree' prefix used across all resources for easy identification

All Provisioned Services

Step 2 — All Services Deployed

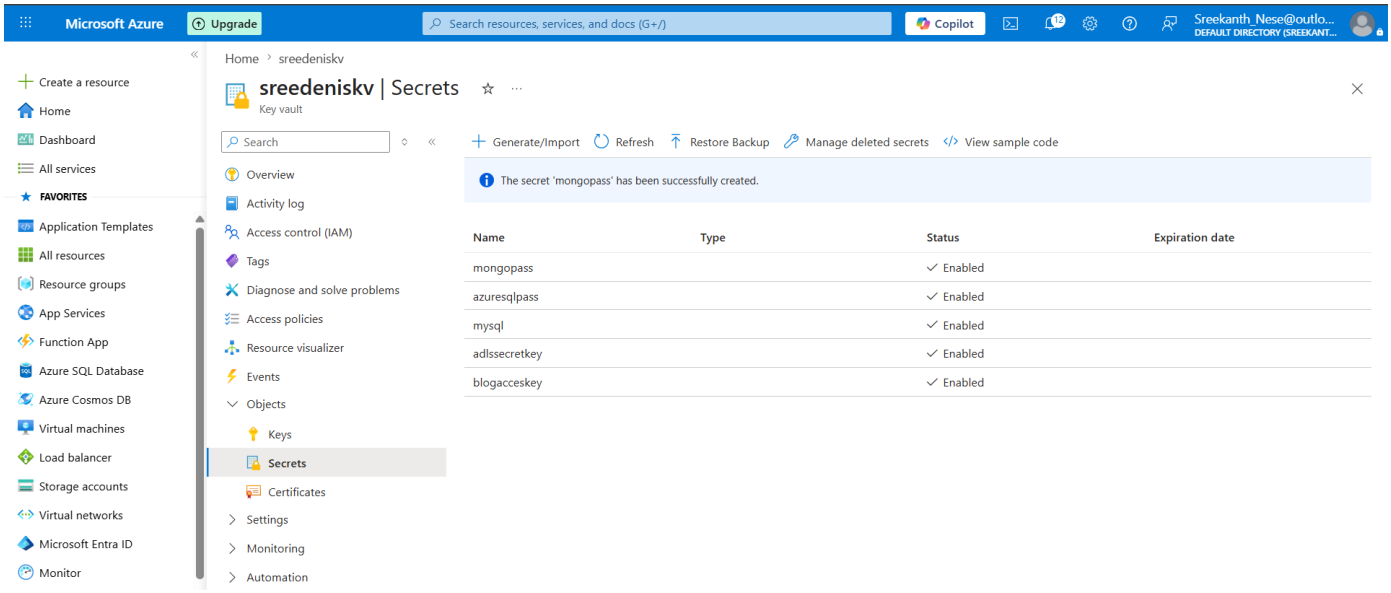


Here's the full stack provisioned for this pipeline: storage account(s) for the data lake, Azure Data Factory for orchestration, Azure SQL Server/Database for serving, and Key Vault for secrets — all deployed in the same region to keep networking simple and latency low.

How it's used: Forms the backbone infrastructure that the rest of the pipeline runs on.

- 6 resources: denisadf123456 (ADF), denisazureblobstorage, denisazureadlsstorage
- denissreeserver / denis_db (SQL), sreedeniskv (Key Vault)

Step 3 — Securing Credentials with Key Vault

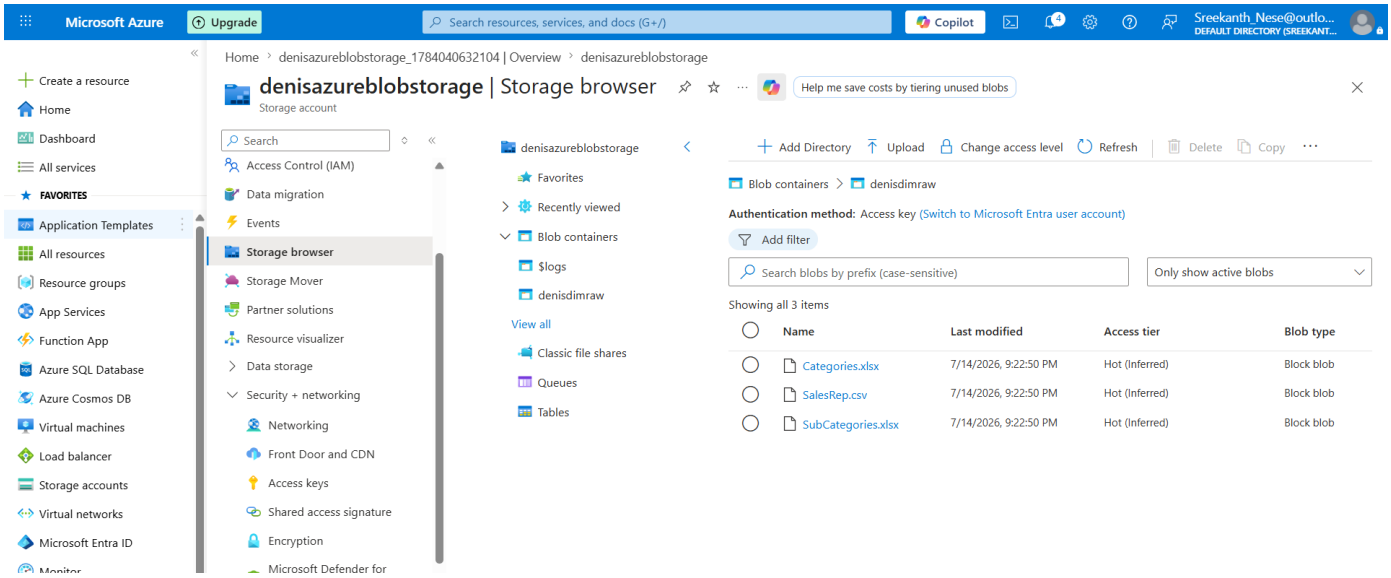


Instead of hardcoding passwords and connection strings inside Data Factory, every secret (storage keys, SQL credentials, DB connection strings) is stored in Key Vault. Access Policies / Azure RBAC restrict who — and which service identity — can read them.

How it's used: Data Factory's Linked Services pull credentials from Key Vault at runtime instead of storing them in plain text — a core data security best practice.

- 5 secrets stored: mongopass, azuresqlpass, mysql, adlssecretkey, blobaccesskey
- All secrets shown as Enabled — no plaintext credentials in ADF

Step 4 — Source Data in Blob Storage

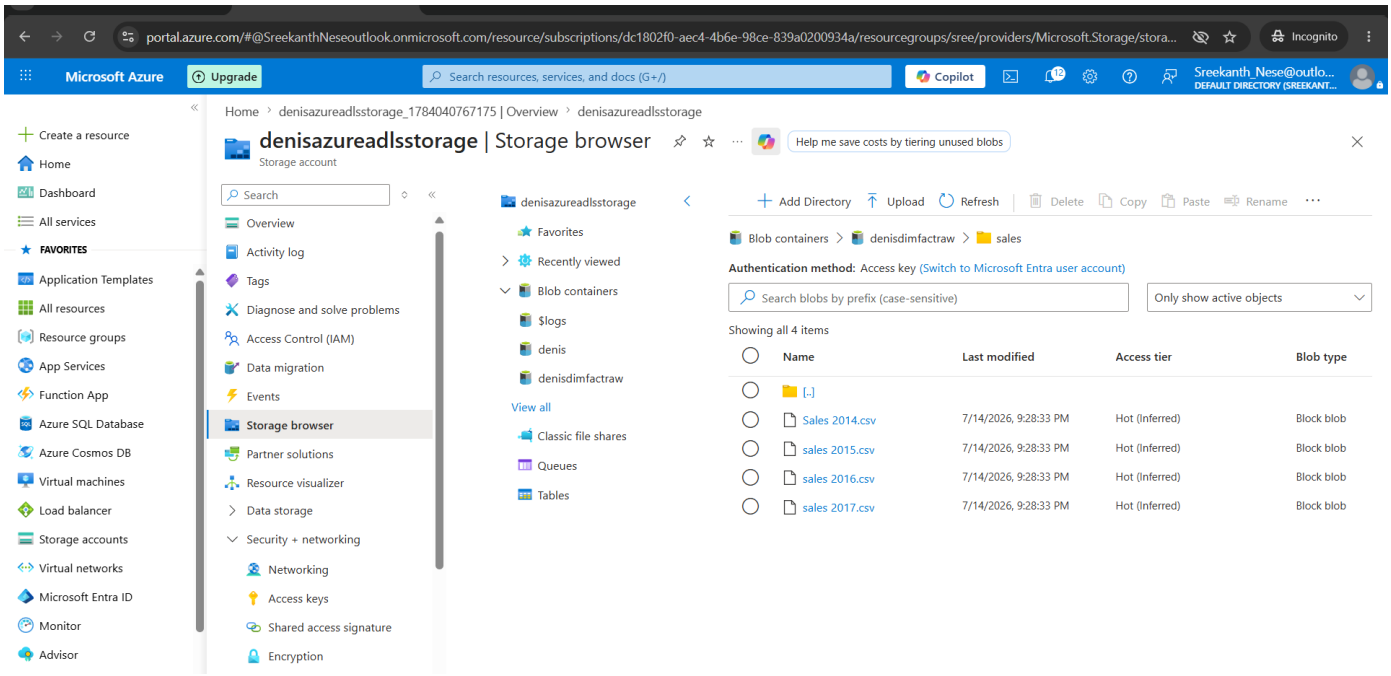


One of the raw data sources — flat CSV/Excel files — sits in Blob Storage, simulating a very common real-world scenario where files land from external vendors or upstream systems before ingestion.

How it's used: Read by an ADF pipeline as a file-based source dataset.

- Container: denisdimraw
- Files: Categories.xlsx, SubCategories.xlsx, SalesRep.csv

Step 5 — Source Data in ADLS Gen2

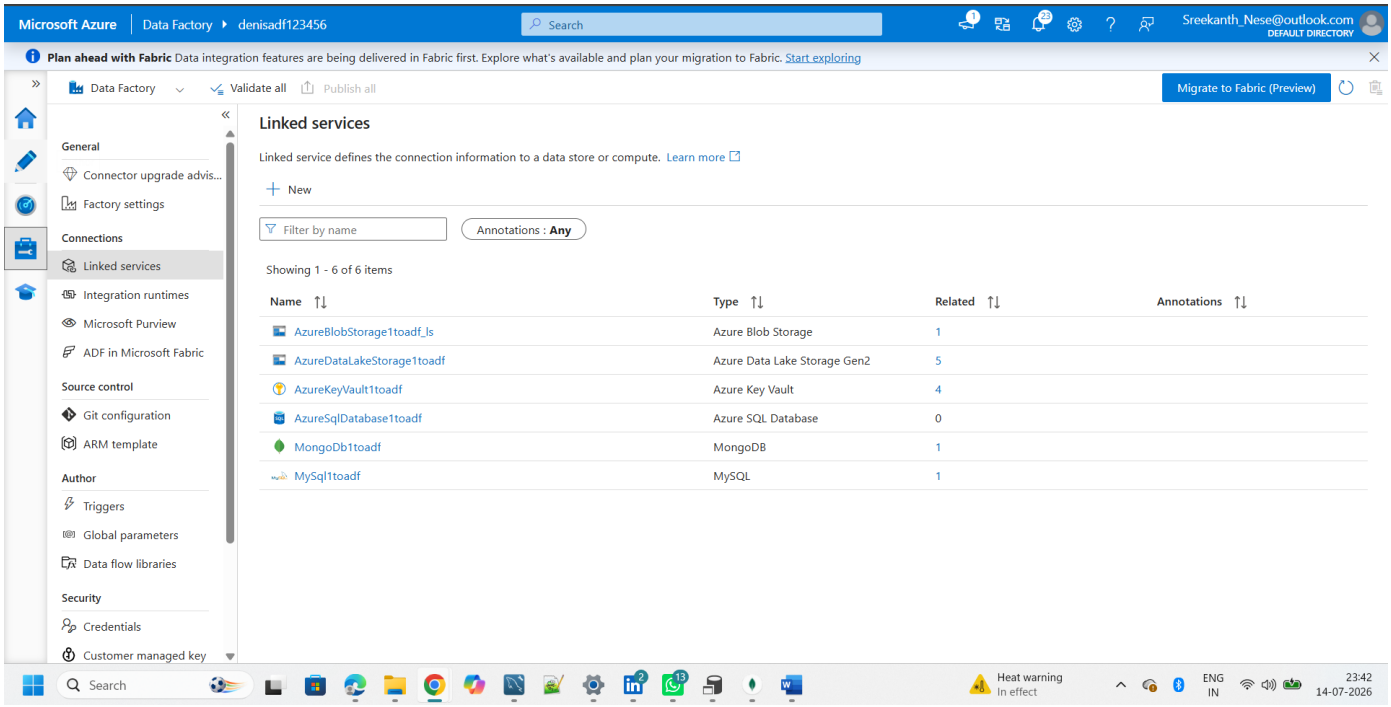


A second file-based source: Azure Data Lake Storage Gen2, which adds a hierarchical namespace on top of Blob Storage for better performance and folder-level access control on big-data workloads. This same storage account also hosts the Bronze/Silver/Gold layers themselves.

How it's used: Acts both as a source system and as the landing zone for all three medallion layers.

- Container: denisdimfactraw/sales
- Files: Sales 2014.csv through Sales 2017.csv

Step 6 — Connecting Every Source (Linked Services)

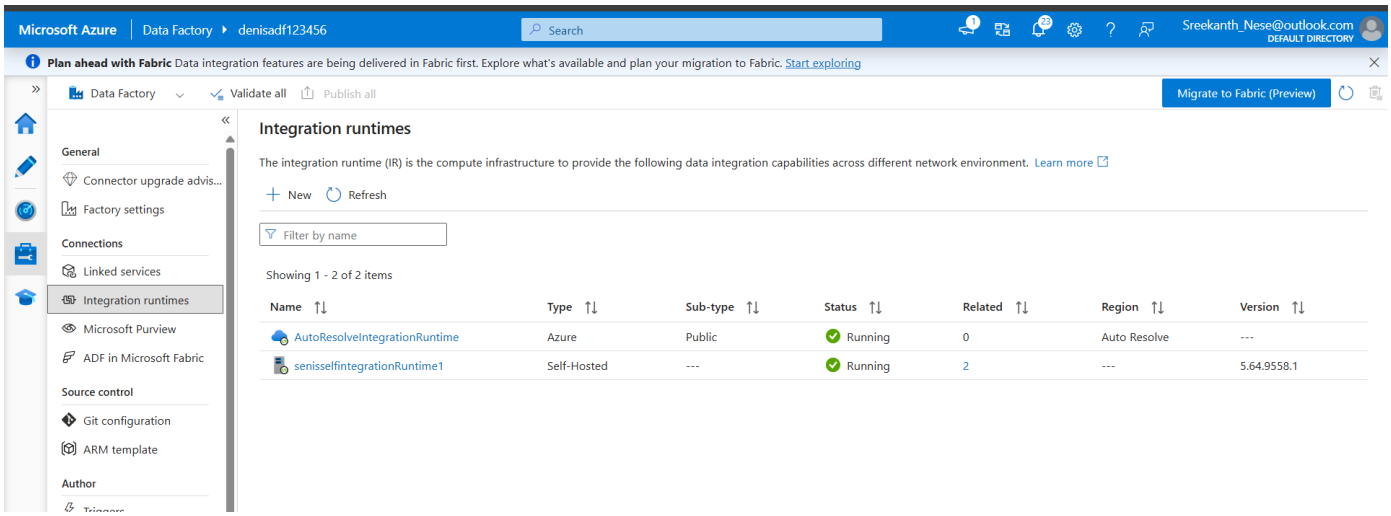


Beyond files, data is also pulled from a relational database (MySQL) and a NoSQL database (MongoDB) — both running **on-premises / outside Azure**, a realistic multi-source scenario. Every source system, storage account, and the SQL Database is registered in Data Factory as a Linked Service — the reusable connection definition ADF uses to reach that system.

How it's used: Linked Services are the foundation every dataset and pipeline activity is built on top of.

- 6 linked services: Blob, ADLS Gen2, Key Vault, Azure SQL DB, MongoDB, MySQL
- Each stores its connection info once and is reused across every pipeline/data flow

Step 7 — Configuring the Integration Runtime

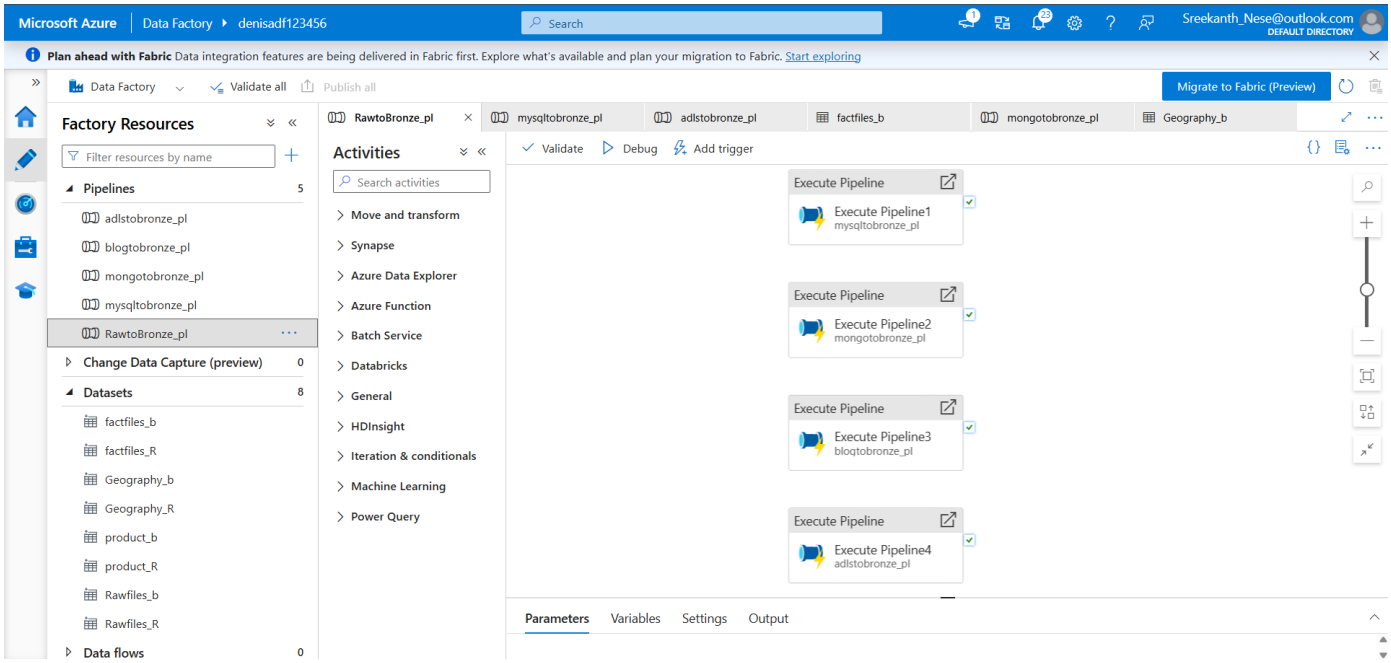


The Integration Runtime is the compute engine behind every Data Factory activity. Blob Storage and ADLS Gen2 use the standard **Azure Integration Runtime** since they're native Azure services. MongoDB and MySQL, being on-premises databases, instead use a **Self-Hosted Integration Runtime (SHIR)** — a lightweight agent installed on a machine with network access to those databases, which lets ADF reach them securely without exposing them to the public internet.

How it's used: Azure IR powers cloud-to-cloud movement; Self-Hosted IR is what makes the on-prem MongoDB/MySQL connections possible at all.

- AutoResolveIntegrationRuntime — Azure, public network
- A second Self-Hosted IR — installed for on-prem DB access

Step 8 — One Pipeline per Source



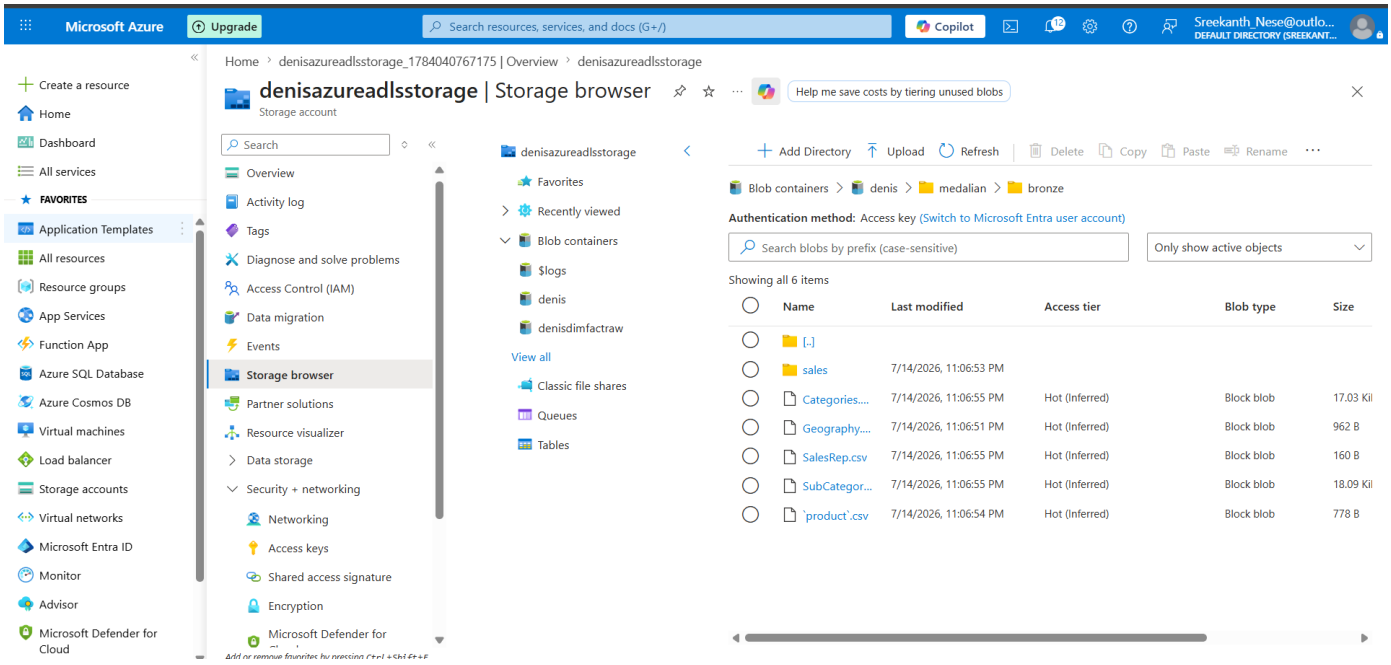
A dedicated pipeline was built for each source system — on-prem MySQL, on-prem MongoDB, Blob CSVs, and ADLS files — to bring them all into the lake. Keeping ingestion modular like this makes each pipeline independently testable and easy to debug when one source fails without breaking the others.

How it's used: Orchestrates the Copy activities that move data from each source into the Bronze layer.

- Pipelines: mysqltobronze_pl, mongotobronze_pl, blobtobronze_pl, adlstobronze_pl
- Each triggered from one master pipeline, in parallel

Bronze Layer

Step 9 — Landing Raw Data in Bronze



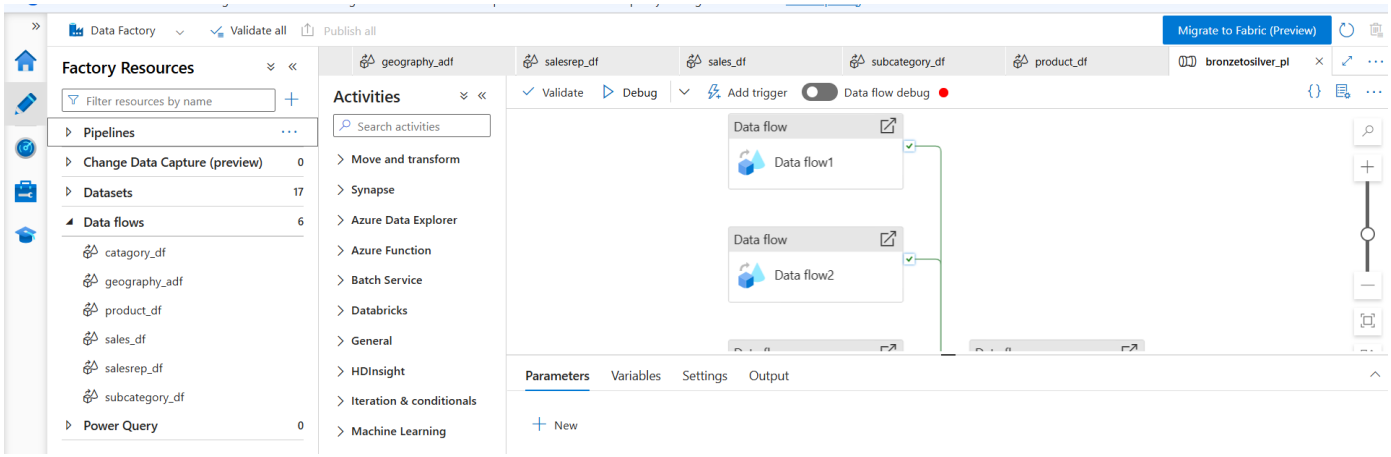
All four sources land here, in the Bronze layer, in their raw / near-original form. Nothing is transformed yet — Bronze is the immutable 'source of truth' copy that allows reprocessing at any time without going back to the original source systems.

How it's used: Acts as the single raw staging zone before any cleansing begins.

- Path: denis/medallion/bronze
- Holds: Categories, Geography, Product, SalesRep, SubCategories, Sales

Mapping Data Flows

Step 10 — Building the Transformation Logic



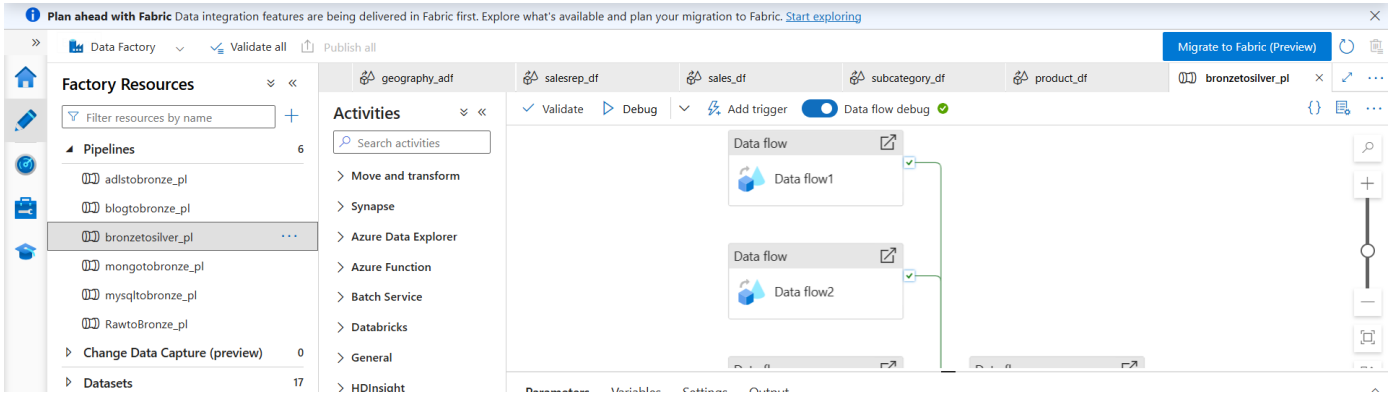
A dedicated Mapping Data Flow was built for each dataset — geography, product, sales, sales-rep, and sub-category — since each needed its own cleansing logic rather than one giant catch-all transformation. Typical logic applied per flow: surrogate key generation, de-duplication, dropping and deriving columns (e.g. splitting location into town/country), and standardizing ID formats.

How it's used: Mapping Data Flows give visual, Spark-powered ETL transformations without writing Spark code by hand.

- 6 flows: category_df, geography_adf, product_df, sales_df, salesrep_df, subcategory_df
- Each flow handles one dataset's cleansing rules independently

Bronze → Silver Pipeline

Step 11 — Orchestrating Bronze to Silver



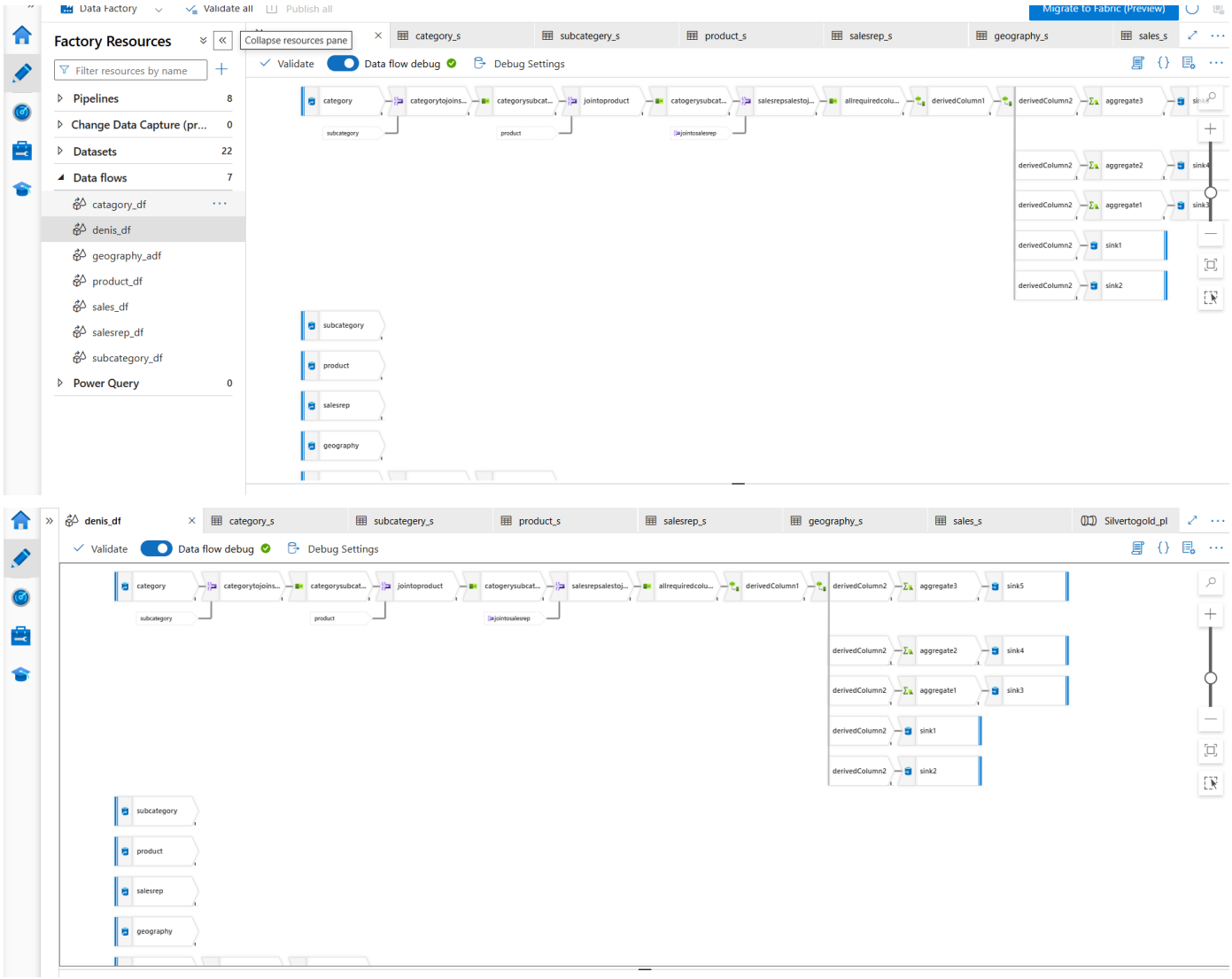
This pipeline chains together the data flows above and writes clean, standardized output to the Silver layer — de-duplicated, keyed, and consistently named. Silver is the 'trusted' layer that downstream jobs and analysts can rely on.

How it's used: Converts five independent transformation flows into one repeatable, schedulable pipeline.

- Pipeline: bronzetosilver_pl
- Runs all 6 transformation data flows in one execution
- Triggered as the second stage in the master pipeline, right after raw ingestion

Silver → Gold Data Flow

Step 12 — Joining & Aggregating into Gold



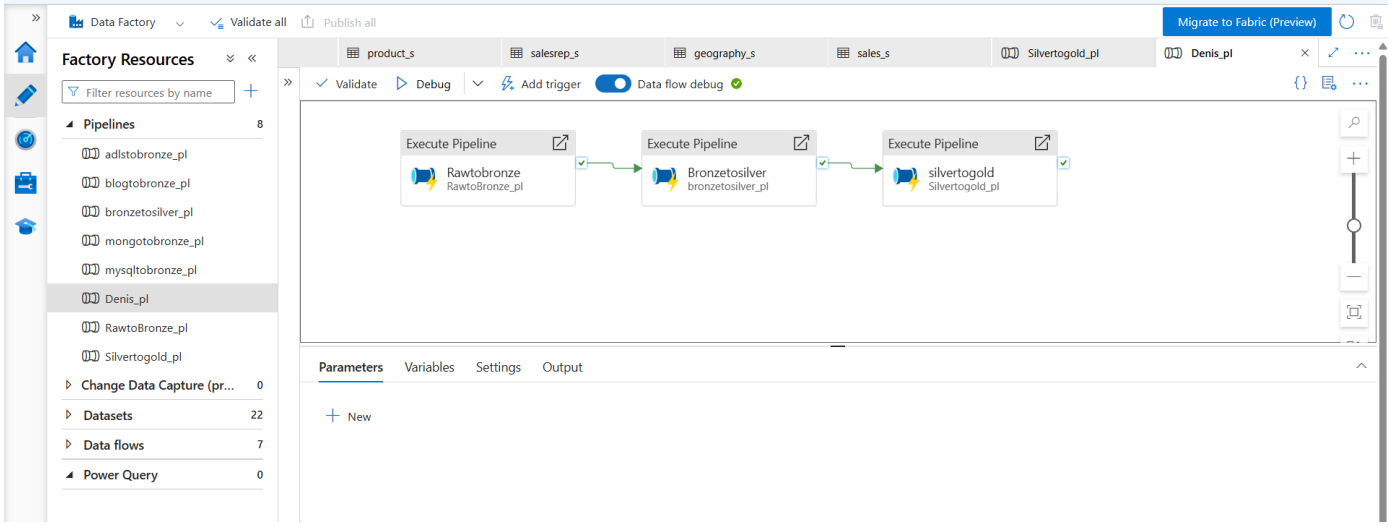
The main data flow joins every Silver dataset together (dimensions + facts) and applies business aggregations, then writes the result to two sinks in the same run: the Gold layer in ADLS and Azure SQL Database — giving both a data-lake copy and a relational copy for different consumption needs.

How it's used: This is where raw, siloed tables finally become one connected, analysis-ready dataset.

- Data flow: denis_df — joins category, subcategory, product, salesrep, geography & sales
- Writes to 5 sinks in parallel: ADLS Gold + Azure SQL Database

Master Orchestration Pipeline

Step 13 — One Pipeline to Run Them All



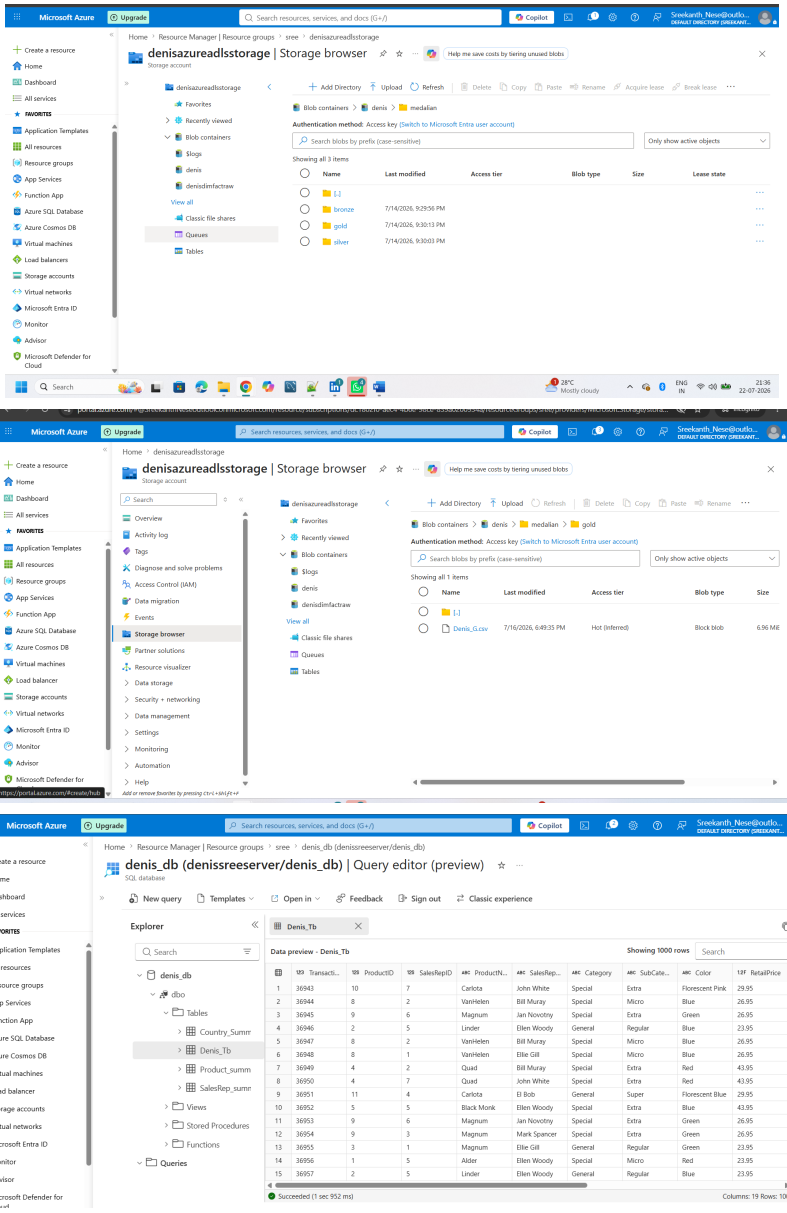
A master pipeline runs every other pipeline in the correct order — sources into Bronze, Bronze into Silver, Silver into Gold — turning several separate jobs into a single, schedulable, end-to-end workflow that can be triggered on demand or on a schedule.

How it's used: This is the pipeline you'd hook up to a trigger for daily/hourly automated runs.

- Master pipeline: Denis_pl
- Sequence: Rawtobronze_pl → Bronzetosilver_pl → Silvertogold_pl
- This is the single pipeline you'd attach a daily/hourly trigger to in production

Step 14 — Final, Business-Ready Data

The finished dataset — clean dimension tables and an aggregated fact table — now lives in both the Gold layer of the data lake and Azure SQL Database, verified and ready to power a BI report (e.g. Power BI) or any downstream analytics consumer.

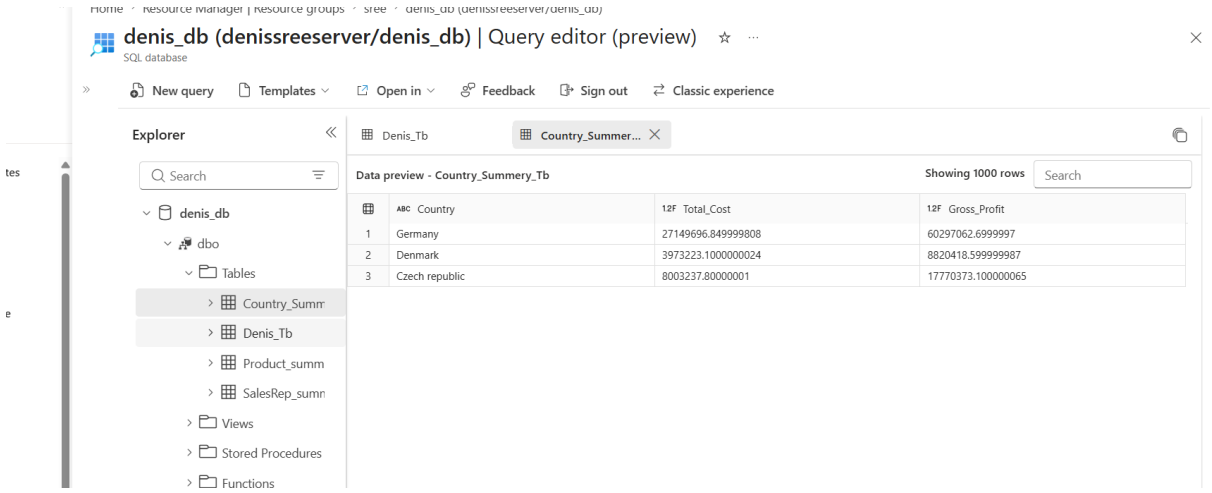


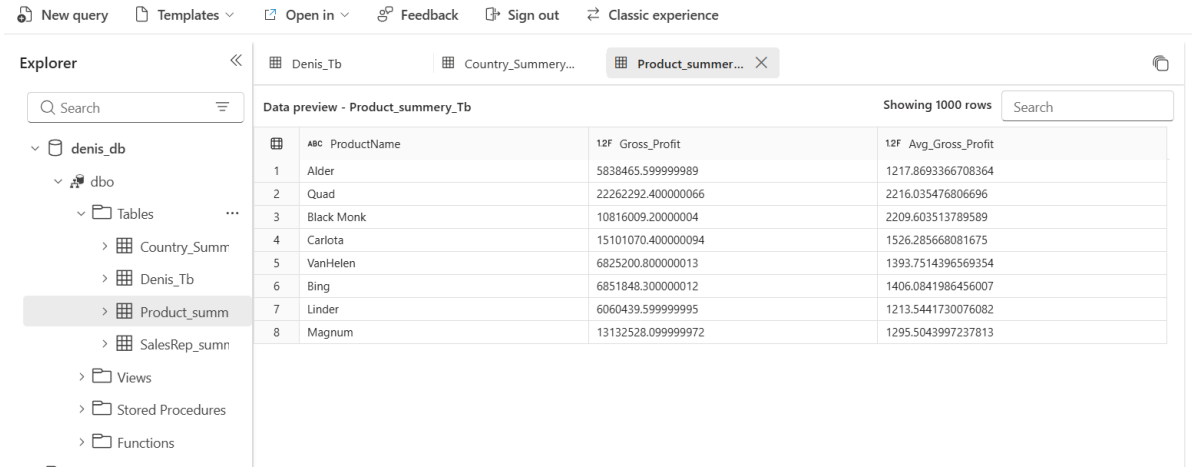
- Gold container path: denis/medallion/gold — verified via Storage Browser

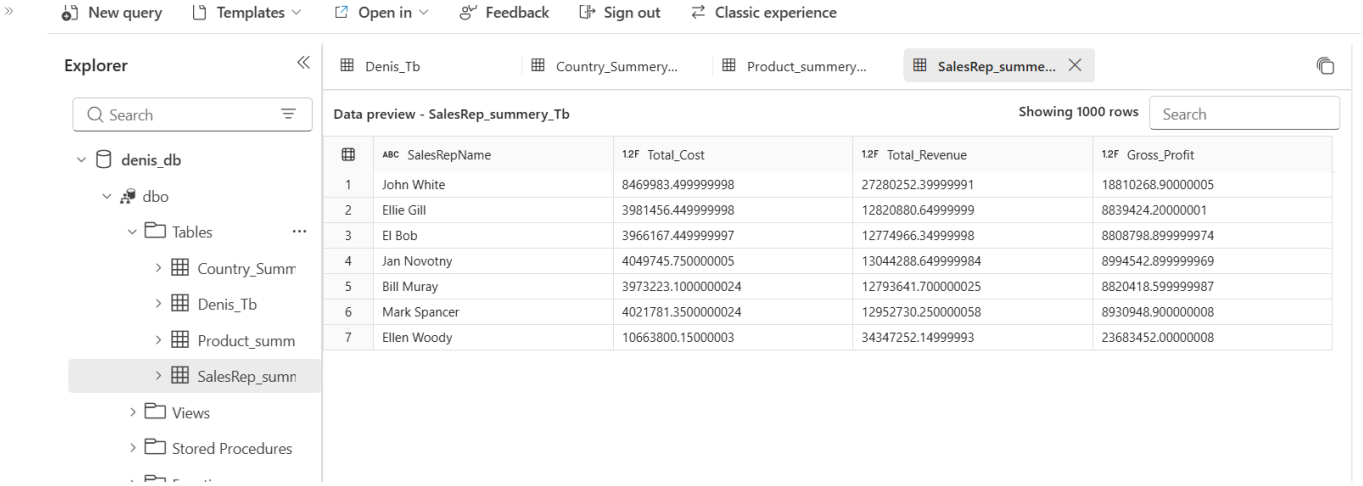
- SQL DB tables queried: Denis_Tb, Country_Summery_Tb, Product_summery_Tb, SalesRep_summary_Tb

Gold Layer & Azure SQL Database (contd.)

A closer look at the query results — verifying row counts, joins, and aggregated totals across each summary table before calling the pipeline complete.







Azure Services Used — Quick Reference

Service	What it is	How it was used here
Resource Group	Logical container for every resource in the project	Organizes storage, ADF, Key Vault and SQL DB as one manageable unit
Blob Storage	General-purpose object storage	Holds raw CSV source files read by Data Factory
ADLS Gen2	Hierarchical, big-data-optimized storage	Source data + hosts the Bronze / Silver / Gold layers
Azure Key Vault	Centralized secrets management	Stores connection strings & credentials used by ADF Linked Services
Azure Data Factory	Cloud ETL/orchestration service	Linked Services, Integration Runtime, Pipelines & Mapping Data Flows for the whole pipeline
Integration Runtime	Compute that executes ADF activities	Runs every Copy activity and Data Flow
Mapping Data Flows	Visual, Spark-based transformation designer	Cleansing (Bronze→Silver) and joins/aggregation (Silver→Gold)
Azure SQL Database	Managed relational database	Serving layer for Gold data, consumable by BI tools
MySQL / MongoDB	On-premises relational & NoSQL source systems	Reached securely from ADF via a Self-Hosted Integration Runtime
Self-Hosted Integration Runtime	Agent bridging Azure and on-prem/private networks	Lets ADF connect to the on-prem MySQL and MongoDB databases